

Object Oriented Programming with JAVA		Semester	3
Course Code	BCS306A	CIE Marks	50
Teaching Hours/Week (L: T:P: S)	2:0:2	SEE Marks	50
Total Hours of Pedagogy	28 Hours of Theory + 20 Hours of Practical	Total Marks	100
Credits	03	Exam Hours	03
Examination type (SEE)	Theory		
Note - Students who have undergone “ Basics of Java Programming-BPLCK105C/205C” in first year are not eligible to opt this course			
Course objectives: - To learn primitive constructs JAVA programming language. - To understand Object Oriented Programming Features of JAVA. - To gain knowledge on: packages, multithreaded programing and exceptions.			
Teaching-Learning Process (General Instructions) These are sample Strategies, which teachers can use to accelerate the attainment of the various course outcomes and make Teaching –Learning more effective - Use Online Java Compiler IDE: https://www.jdoodle.com/online-java-compiler/ or any other. - Demonstration of programing examples. - Chalk and board, power point presentations - Online material (Tutorials) and video lectures.			
Module-1 An Overview of Java: Object-Oriented Programming (Two Paradigms, Abstraction, The Three OOP Principles), Using Blocks of Code, Lexical Issues (Whitespace, Identifiers, Literals, Comments, Separators, The Java Keywords). Data Types, Variables, and Arrays: The Primitive Types (Integers, Floating-Point Types, Characters, Booleans), Variables, Type Conversion and Casting, Automatic Type Promotion in Expressions, Arrays, Introducing Type Inference with Local Variables. Operators: Arithmetic Operators, Relational Operators, Boolean Logical Operators, The Assignment Operator, The ? Operator, Operator Precedence, Using Parentheses. Control Statements: Java’s Selection Statements (if, The Traditional switch), Iteration Statements (while, do-while, for, The For-Each Version of the for Loop, Local Variable Type Inference in a for Loop, Nested Loops), Jump Statements (Using break, Using continue, return). Chapter 2, 3, 4, 5			
Module-2 Introducing Classes: Class Fundamentals, Declaring Objects, Assigning Object Reference Variables, Introducing Methods, Constructors, The this Keyword, Garbage Collection. Methods and Classes: Overloading Methods, Objects as Parameters, Argument Passing, Returning Objects, Recursion, Access Control, Understanding static, Introducing final, Introducing Nested and Inner Classes. Chapter 6, 7			
Module-3 Inheritance: Inheritance Basics, Using super, Creating a Multilevel Hierarchy, When Constructors Are Executed, Method Overriding, Dynamic Method Dispatch, Using Abstract Classes, Using final with Inheritance, Local Variable Type Inference and Inheritance, The Object Class. Interfaces: Interfaces, Default Interface Methods, Use static Methods in an Interface, Private Interface Methods. Chapter 8, 9			

	Module-4
	Packages: Packages, Packages and Member Access, Importing Packages. Exceptions: Exception-Handling Fundamentals, Exception Types, Uncaught Exceptions, Using try and catch, Multiple catch Clauses, Nested try Statements, throw, throws, finally, Java's Built-in Exceptions, Creating Your Own Exception Subclasses, Chained Exceptions. Chapter 9, 10
	Module-5
	Multithreaded Programming: The Java Thread Model, The Main Thread, Creating a Thread, Creating Multiple Threads, Using <code>isAlive()</code> and <code>join()</code>, Thread Priorities, Synchronization, Interthread Communication, Suspending, Resuming, and Stopping Threads, Obtaining a Thread's State. Enumerations, Type Wrappers and Autoboxing: Enumerations (Enumeration Fundamentals, The <code>values()</code> and <code>valueOf()</code> Methods), Type Wrappers (Character, Boolean, The Numeric Type Wrappers), Autoboxing (Autoboxing and Methods, Autoboxing/Unboxing Occurs in Expressions, Autoboxing/Unboxing Boolean and Character Values). Chapter 11, 12
Course outcome (Course Skill Set) At the end of the course, the student will be able to: 1. Demonstrate proficiency in writing simple programs involving branching and looping structures. 2. Design a class involving data members and methods for the given scenario. 3. Apply the concepts of inheritance and interfaces in solving real world problems. 4. Use the concept of packages and exception handling in solving complex problem 5. Apply concepts of multithreading, autoboxing and enumerations in program development	
Programming Experiments (Suggested and are not limited to) 1. Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments). 2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations. 3. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method <code>raiseSalary</code> (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration. 4. A class called MyPoint, which models a 2D point with x and y coordinates, is designed as follows: • Two instance variables x (int) and y (int). • A default (or "no-arg") constructor that construct a point at the default location of (0, 0). • A overloaded constructor that constructs a point with the given x and y coordinates. • A method <code>setXY()</code> to set both x and y. • A method <code>getXY()</code> which returns the x and y in a 2-element int array. • A <code>toString()</code> method that returns a string description of the instance in the format "(x, y)". • A method called <code>distance(int x, int y)</code> that returns the distance from this point to another point at the given (x, y) coordinates • An overloaded <code>distance(MyPoint another)</code> that returns the distance from this point to the given MyPoint instance (called another) • Another overloaded <code>distance()</code> method that returns the distance from this point to the origin (0,0) Develop the code for the class MyPoint. Also develop a JAVA program (called TestMyPoint) to test all the methods defined in the class. 5. Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named <code>draw ()</code> and <code>erase ()</code>. Demonstrate	

- polymorphism concepts by developing suitable methods, defining member data and main program.
6. Develop a JAVA program to create an abstract class Shape with abstract methods calculateArea() and calculatePerimeter(). Create subclasses Circle and Triangle that extend the Shape class and implement the respective methods to calculate the area and perimeter of each shape.
 7. Develop a JAVA program to create an interface Resizable with methods resizeWidth(int width) and resizeHeight(int height) that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods
 8. Develop a JAVA program to create an outer class with a function display. Create another class inside the outer class named inner with a function called display and call the two functions in the main class.
 9. Develop a JAVA program to raise a custom exception (user defined exception) for DivisionByZero using try, catch, throw and finally.
 10. Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.
 11. Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds).
 12. Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the theory component of the IPCC (maximum marks 50)

- IPCC means practical portion integrated with the theory of the course.
- CIE marks for the theory component are **25 marks** and that for the practical component is **25 marks**.
- 25 marks for the theory component are split into **15 marks** for two Internal Assessment Tests (Two Tests, each of 15 Marks with 01-hour duration, are to be conducted) and **10 marks** for other assessment methods mentioned in 22OB4.2. The first test at the end of 40-50% coverage of the syllabus and the second test after covering 85-90% of the syllabus.
- Scaled-down marks of the sum of two tests and other assessment methods will be CIE marks for the theory component of IPCC (that is for **25 marks**).
- The student has to secure 40% of 25 marks to qualify in the CIE of the theory component of IPCC.

CIE for the practical component of the IPCC

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.
- The laboratory test (**duration 02/03 hours**) after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks**.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

SEE for IPCC

Theory SEE will be conducted by University as per the scheduled timetable, with common question papers for the course (**duration 03 hours**)

1. The question paper will have ten questions. Each question is set for 20 marks.
2. There will be 2 questions from each module. Each of the two questions under a module (with a maximum of 3 sub-questions), **should have a mix of topics** under that module.
3. The students have to answer 5 full questions, selecting one full question from each module.
4. Marks scored by the student shall be proportionally scaled down to 50 Marks

The theory portion of the IPCC shall be for both CIE and SEE, whereas the practical portion will have a CIE component only. Questions mentioned in the SEE paper may include questions from the practical component.

Suggested Learning Resources:

Textbook

1. Java: The Complete Reference, Twelfth Edition, by Herbert Schildt, November 2021, McGraw-Hill, ISBN: 9781260463422

Reference Books

1. Programming with Java, 6th Edition, by E Balagurusamy, Mar-2019, McGraw Hill Education, ISBN: 9789353162337.
2. Thinking in Java, Fourth Edition, by Bruce Eckel, Prentice Hall, 2006
(https://sd.blackball.lv/library/thinking_in_java_4th_edition.pdf)

Web links and Video Lectures (e-Resources):

- Java Tutorial: <https://www.geeksforgeeks.org/java/>
- Introduction To Programming In Java (by Evan Jones, Adam Marcus and Eugene Wu):
<https://ocw.mit.edu/courses/6-092-introduction-to-programming-in-java-january-iap-2010/>
- Java Tutorial: <https://www.w3schools.com/java/>
- Java Tutorial: <https://www.javatpoint.com/java-tutorial>

JAVA LAB PROGRAMS

1. Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).

```
import java.util.Scanner;

public class MatrixAddition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input for matrix size N
        System.out.print("Enter the order N of the matrices: ");
        int n = sc.nextInt();
        int[][] matrix1 = new int[n][n];
        int[][] matrix2 = new int[n][n];
        int[][] result = new int[n][n];

        // Input for first matrix
        System.out.println("Enter elements for Matrix 1:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print("Element [" + i + "][" + j + "]: ");
                matrix1[i][j] = sc.nextInt();
            }
        }

        // Input for second matrix
        System.out.println("Enter elements for Matrix 2:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print("Element [" + i + "][" + j + "]: ");
                matrix2[i][j] = sc.nextInt();
            }
        }

        // Adding the matrices
        for (int i = 0; i < n; i++) {
```

```
for (int j = 0; j < n; j++) {  
    result[i][j] = matrix1[i][j] + matrix2[i][j];  
}  
}  
  
// Displaying the result  
System.out.println("\nResultant Matrix after Addition:");  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.print(result[i][j] + "\t");  
    }  
    System.out.println();  
}  
sc.close();  
}  
}
```

OUTPUT:

2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations.

```
import java.util.Scanner;

// Stack class
class Stack {
    private int maxSize = 10; // Maximum stack size
    private int[] stackArray;
    private int top;

    // Constructor
    public Stack() {
        stackArray = new int[maxSize];
        top = -1;
    }

    // Push method
    public void push(int value) {
        if (top == maxSize - 1) {
            System.out.println("Stack Overflow! Cannot push " + value);
        } else {
            stackArray[++top] = value;
            System.out.println(value + " pushed to stack.");
        }
    }

    // Pop method
    public int pop() {
        if (top == -1) {
            System.out.println("Stack Underflow! Stack is empty.");
            return -1;
        } else {
            int value = stackArray[top--];
            System.out.println(value + " popped from stack.");
            return value;
        }
    }
}
```

```
}  
}  
  
// Display method  
public void display() {  
    if (top == -1) {  
        System.out.println("Stack is empty.");  
    } else {  
        System.out.println("Stack elements are (Top to Bottom):");  
        for (int i = top; i >= 0; i--) {  
            System.out.println(stackArray[i]);  
        }  
    }  
}  
  
// Main class  
public class StackDemo {  
    public static void main(String[] args) {  
        Stack stack = new Stack();  
        Scanner sc = new Scanner(System.in);  
        int choice, value;  
        do {  
            System.out.println("\n==== Stack Operations Menu ====");  
            System.out.println("1. Push");  
            System.out.println("2. Pop");  
            System.out.println("3. Display");  
            System.out.println("4. Exit");  
            System.out.print("Enter your choice (1-4): ");  
            choice = sc.nextInt();  
            switch (choice) {  
                case 1:  
                    System.out.print("Enter value to push: ");
```

```
value = sc.nextInt();
stack.push(value);
break;
case 2:
stack.pop();
break;
case 3:
stack.display();
break;
case 4:
System.out.println("Exiting the program...");
break;
default:
System.out.println("Invalid choice! Please enter between 1 to 4.");
}
} while (choice != 4);
sc.close();
}
}
```

OUTPUT:

```
Enter your choice (1-4): 1
Enter value to push: 20
20 pushed to stack.

==== Stack Operations Menu ====
1. Push
2. Pop
3. Display
4. Exit
Enter your choice (1-4): 1
Enter value to push: 30
30 pushed to stack.

==== Stack Operations Menu ====
1. Push
2. Pop
3. Display
4. Exit
Enter your choice (1-4): 1
Enter value to push: 40
40 pushed to stack.

==== Stack Operations Menu ====
1. Push
2. Pop
3. Display
4. Exit
Enter your choice (1-4): 3
Stack elements are (Top to Bottom):
40
30
20

==== Stack Operations Menu ====
1. Push
2. Pop
3. Display
4. Exit
Enter your choice (1-4): 2
40 popped from stack.

==== Stack Operations Menu ====
1. Push
2. Pop
3. Display
4. Exit
Enter your choice (1-4): 3
Stack elements are (Top to Bottom):
30
20

==== Stack Operations Menu ====
1. Push
2. Pop
3. Display
4. Exit
Enter your choice (1-4):
```

<

3. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raise Salary (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration.

```
import java.util.Scanner;

// Employee class
class Employee {
    private int id;
    private String name;
    private double salary;
    // Constructor to initialize employee details
    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
    // Method to display employee details
    public void display() {
        System.out.println("-----");
        System.out.println("Employee ID : " + id);
        System.out.println("Employee Name : " + name);
        System.out.println("Employee Salary: ₹" + salary);
        System.out.println("-----");
    }
    // Method to raise salary by a given percentage
    public void raiseSalary(double percent) {
        if (percent > 0) {
            double increment = salary * percent / 100;
            salary += increment;
            System.out.println("Salary increased by " + percent + "% (₹" + increment + ")");
        } else {
            System.out.println("Invalid percentage. Salary not changed.");
        }
    }
}
```

```
}  
}  
}  
  
// Main class  
public class EmployeeDemo {  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    // Input employee details  
    System.out.print("Enter Employee ID: ");  
    int id = sc.nextInt();  
    sc.nextLine(); // Clear input buffer  
    System.out.print("Enter Employee Name: ");  
    String name = sc.nextLine();  
    System.out.print("Enter Employee Salary: ");  
    double salary = sc.nextDouble();  
    // Create Employee object  
    Employee emp = new Employee(id, name, salary);  
    // Display initial details  
    System.out.println("\n--- Employee Details Before Salary Raise ---");  
    emp.display();  
    // Input raise percentage  
    System.out.print("\nEnter percentage to raise salary: ");  
    double percent = sc.nextDouble();  
    // Apply raise  
    emp.raiseSalary(percent);  
    // Display updated details  
    System.out.println("\n--- Employee Details After Salary Raise ---");  
    emp.display();  
    sc.close();  
}  
}
```

OUTPUT:

4. A class called MyPoint, which models a 2D point with x and y coordinates, is designed as follows:

- Two instance variables x (int) and y (int).
- A default (or "no-arg") constructor that construct a point at the default location of (0, 0).
- A overloaded constructor that constructs a point with the given x and y coordinates.
- A method setXY() to set both x and y.
- A method getXY() which returns the x and y in a 2-element int array.
- A toString() method that returns a string description of the instance in the format "(x, y)".
- A method called distance(int x, int y) that returns the distance from this point to another point at the given (x, y) coordinates
- An overloaded distance(MyPoint another) that returns the distance from this point to the given MyPoint instance (called another)
- Another overloaded distance() method that returns the distance from this point to the origin (0,0)

Develop the code for the class MyPoint. Also develop a JAVA program (called TestMyPoint) to test all the methods defined in the class.

1. MyPoint.java

```
public class MyPoint {  
    // Instance variables  
    private int x;  
    private int y;  
    // Default constructor (0, 0)  
    public MyPoint() {  
        this.x = 0;  
        this.y = 0;  
    }  
    // Overloaded constructor with parameters  
    public MyPoint(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    // Setter method to set both x and y  
    public void setXY(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```



```
}  
// Getter method that returns x and y in an array  
public int[] getXY() {  
    return new int[] { x, y };  
}  
// toString method  
public String toString() {  
    return "(" + x + ", " + y + ")";  
}  
// Method to calculate distance to another point with (x, y)  
public double distance(int x, int y) {  
    int dx = this.x - x;  
    int dy = this.y - y;  
    return Math.sqrt(dx * dx + dy * dy);  
}  
// Overloaded distance method to another MyPoint object  
public double distance(MyPoint another) {  
    int dx = this.x - another.x;  
    int dy = this.y - another.y;  
    return Math.sqrt(dx * dx + dy * dy);  
}  
// Overloaded distance method to the origin (0, 0)  
public double distance() {  
    return Math.sqrt(x * x + y * y);  
}  
}
```

2. TestMyPoint.java

```
public class TestMyPoint {  
    public static void main(String[] args) {  
        // Create points
```

```
MyPoint p1 = new MyPoint(); // Default (0,0)
MyPoint p2 = new MyPoint(3, 4); // (3,4)
// Test toString()
System.out.println("p1: " + p1); // (0, 0)
System.out.println("p2: " + p2); // (3, 4)
// Test setXY()
p1.setXY(5, 6);
System.out.println("After setXY, p1: " + p1); // (1, 2)
// Test getX()
int[] coords = p1.getX();
System.out.println("p1 x: " + coords[0] + ", y: " + coords[1]);
// Test distance(x, y)
double d1 = p1.distance(4, 6);
System.out.println("Distance from p1 to (4,6): " + d1);
// Test distance(MyPoint another)
double d2 = p1.distance(p2);
System.out.println("Distance from p1 to p2: " + d2);
// Test distance() to origin
double d3 = p2.distance();
System.out.println("Distance from p2 to origin: " + d3);
}
}
```

OUTPUT:

5. Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program.

```
// Base class
class Shape {
    void draw() {
        System.out.println("Drawing Shape");
    }
    void erase() {
        System.out.println("Erasing Shape");
    }
}

// Subclass Circle
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
    void erase() {
        System.out.println("Erasing Circle");
    }
}

// Subclass Triangle
class Triangle extends Shape {
    void draw() {
        System.out.println("Drawing Triangle");
    }
    void erase() {
        System.out.println("Erasing Triangle");
    }
}

// Subclass Square
class Square extends Shape {
    void draw() {
        System.out.println("Drawing Square");
    }
}
```

```
    }  
    void erase() {  
        System.out.println("Erasing Square");  
    }  
}  
  
// Main class to demonstrate polymorphism  
public class ShapeDemo {  
    public static void main(String[] args) {  
        // Create shape references pointing to different objects  
        Shape shape;  
        shape = new Circle();  
        shape.draw();  
        shape.erase();  
        shape = new Triangle();  
        shape.draw();  
        shape.erase();  
        shape = new Square();  
        shape.draw();  
        shape.erase();  
    }  
}
```

OUTPUT:

6. Develop a JAVA program to create an abstract class Shape with abstract methods calculate Area() and Calculate Perimeter(). Create subclasses Circle and Triangle that extend the Shape class and implement the respective methods to calculate the area and perimeter of each shape.

```
abstract class Shape1 {  
    abstract void area();  
    abstract void perimeter();  
}  
  
class Circle1 extends Shape1 {  
    double r;  
    Circle1(double radius) {  
        r = radius;  
    }  
    void area() {  
        System.out.println("Area of Circle: " + (3.14 * r * r));  
    }  
    void perimeter() {  
        System.out.println("Perimeter of Circle: " + (2 * 3.14 * r));  
    }  
}  
  
class Triangle1 extends Shape1 {  
    double a, b, c;  
    Triangle1(double x, double y, double z) {  
        a = x;  
        b = y;  
        c = z;  
    }  
    void area() {  
        double s = (a + b + c) / 2;  
        double ar = Math.sqrt(s * (s - a) * (s - b) * (s - c));  
        System.out.println("Area of Triangle: " + ar);  
    }  
}
```

```
void perimeter() {  
    System.out.println("Perimeter of Triangle: " + (a + b + c));  
}  
  
public class ShapeTest {  
    public static void main(String[] args) {  
        Circle1 c = new Circle1(5);  
        c.area();  
        c.perimeter();  
        System.out.println();  
        Triangle1 t = new Triangle1(3, 4, 5);  
        t.area();  
        t.perimeter();  
    }  
}
```

OUTPUT:

7. Develop a JAVA program to create an interface Resizable with methods `resize Width(int width)` and `resize Height(int height)` that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods.

// Interface Resizable

```
interface Resizable {  
void resizeWidth(int width);  
void resizeHeight(int height);  
}
```

// Rectangle class implementing Resizable

```
class Rectangle implements Resizable {  
int width;  
int height;
```

// Constructor

```
Rectangle(int w, int h) {  
    width = w;  
    height = h;  
}
```

// Method to display current width and height

```
void display() {  
    System.out.println("Width: " + width + ", Height: " + height);  
}
```

// Implement `resizeWidth` method

```
public void resizeWidth(int newWidth) {  
    width = newWidth;  
}
```

// Implement `resizeHeight` method

```
public void resizeHeight(int newHeight) {  
    height = newHeight;  
}  
}
```

```
// Main class
public class ResizeTest {
    public static void main(String[] args) {
        // Create rectangle object
        Rectangle rect = new Rectangle(10, 20);
        System.out.println("Original Size:");
        rect.display();
        // Resize width and height
        rect.resizeWidth(30);
        rect.resizeHeight(40);
        System.out.println("Resized Size:");
        rect.display();
    }
}
```

OUTPUT:

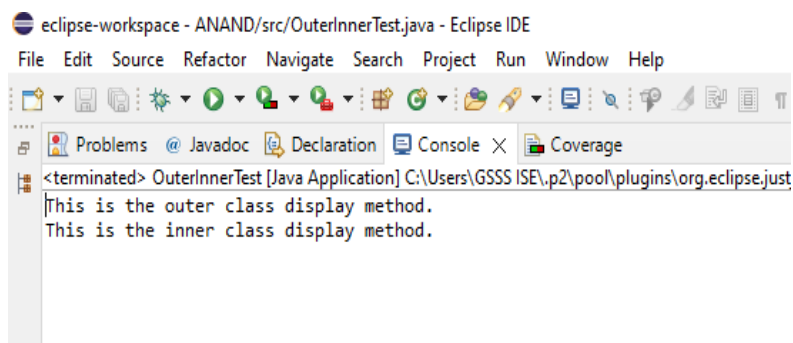
8. Develop a JAVA program to create an outer class with a function display. Create another class inside the outer class named inner with a function called display and call the two functions in the main class.

```
// Outer class
class Outer {
    void display() {
        System.out.println("This is the outer class display method.");
    }
}

// Inner class
class Inner {
    void display() {
        System.out.println("This is the inner class display method.");
    }
}

// Main class
public class OuterInnerTest {
    public static void main(String[] args) {
        // Create outer class object
        Outer outer = new Outer();
        outer.display(); // Call outer class display
        // Create inner class object using outer class object
        Outer.Inner inner = outer.new Inner();
        inner.display(); // Call inner class display
    }
}
```

OUTPUT:



9. Develop a JAVA program to raise a custom exception (user defined exception) for Division By Zero using try, catch, throw and finally.

// Custom Exception class

class DivisionByZeroException **extends** Exception {

// Constructor

DivisionByZeroException(String message) {

super(message);

}

}

// Main class

public class DivisionTest {

public static void main(String[] args) {

int num = 10;

int den = 0; // Deliberately setting denominator to zero

try {

if (den == 0) {

throw new DivisionByZeroException("Division by zero is not allowed!");

}

int result = num / den;

System.**out**.println("Result: " + result);

}

catch (DivisionByZeroException e) {

System.**out**.println("Exception Caught: " + e.getMessage());

}

finally {

System.**out**.println("Finally block executed.");

}

}

}

OUTPUT:

10. Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.

✓ Step 1: Create the Package (mypack)

File: mypack/Message.java

```
package mypack; // Declare package name
// Class inside the package
public class Message {
    public void display() {
        System.out.println("Hello from mypack!");
    }
}
```

✓ Step 2: Create the Main Class to Use the Package

File: TestPackage.java

```
import mypack.Message; // Import the class from mypack
public class TestPackage {
    public static void main(String[] args) {
        Message msg = new Message();
        msg.display();
    }
}
```

11. Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds).

```
// Create a class that implements Runnable
class MyThread implements Runnable {
    String threadName;

    // Constructor
    MyThread(String name) {
        threadName = name;
    }

    // The run method contains the code that runs in the thread
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(threadName + " - Count: " + i);
            try {
                Thread.sleep(500); // Suspend thread for 500 milliseconds
            } catch (InterruptedException e) {
                System.out.println(threadName + " interrupted.");
            }
        }
        System.out.println(threadName + " finished.");
    }
}

// Main class
public class RunnableThreadExample {
    public static void main(String[] args) {
        // Create Runnable objects
        MyThread t1 = new MyThread("Thread-1");
        MyThread t2 = new MyThread("Thread-2");
        // Create Thread objects with Runnable
        Thread thread1 = new Thread(t1);
```

```
Thread thread2 = new Thread(t2);
```

```
// Start the threads
```

```
thread1.start();
```

```
thread2.start();
```

```
}
```

```
}
```

OUTPUT:

12. Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

```
class MyThread1 extends Thread {
    MyThread1() {
        super("ChildThread"); // Call Thread class constructor to set thread name
        start(); // Start the thread
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + ": " + i);
            try {
                Thread.sleep(500); // Pause for 500 milliseconds
            } catch (Exception e) {
                System.out.println(e);
            }
        }
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread1 t = new MyThread1(); // Create and start child thread
        // Main thread work
        for (int i = 1; i <= 5; i++) {
            System.out.println("Main Thread: " + i);
            try {
                Thread.sleep(500); // Pause for 500 milliseconds
            } catch (Exception e) {
                System.out.println(e);
            }
        }
    }
}
```

```
}  
}
```

OUTPUT:

VIVA QUESTIONS:

- **Matrix Addition – Command Line Arguments**

1. **What is the use of command-line arguments in Java?**
→It allows passing values to the main() method at runtime.
 2. **Syntax of command-line arguments in Java?**
→public static void main(String[] args)
 3. **How to convert command-line arguments to integers?**
→Using Integer.parseInt(args[index]).
 4. **What happens if no arguments are provided?**
→It throws ArrayIndexOutOfBoundsException.
-

- **Stack Class**

5. **What is a stack?**
→Stack is a linear data structure that follows **LIFO (Last In, First Out)** principle.
 6. **Key operations in stack?**
→push() (add), pop() (remove), display().
 7. **What happens when stack overflows?**
→Adding an element to a full stack causes **Stack Overflow**.
 8. **What happens when stack underflows?**
→Removing an element from an empty stack causes **Stack Underflow**.
-

- **Employee Class – OOP Concepts**

9. **What is encapsulation?**
→Wrapping data (variables) and methods into a single unit (class).
 10. **How do you implement encapsulation?**
→Using private variables and public getters/setters.
 11. **What is the use of the raiseSalary() method?**
→It increases the employee's salary by a given percentage.
-

- **MyPoint Class – Method Overloading**

12. What is method overloading?

→Defining multiple methods with the same name but different parameters.

13. How is distance calculated between two points?

→Using the formula

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

14. What does the toString() method do?

→Returns the string representation of the object.

- **Polymorphism with Shape, Circle, Triangle, Square**

15. What is polymorphism?

→The ability of one interface to be used for different data types.

16. What is method overriding?

→Redefining a method of the parent class in the subclass.

17. What is dynamic polymorphism?

→When method calls are resolved at **runtime** using method overriding.

- **Abstract Class – Shape**

18. What is an abstract class?

→A class that cannot be instantiated and may contain abstract methods.

19. What is an abstract method?

→A method without a body that must be overridden by subclasses.

20. Why do we use abstract classes?

→To provide a base for other classes with common behavior but force implementation of certain methods.

- **Interface – Resizable**

21. What is an interface?

→A collection of abstract methods that a class must implement.

22. Difference between abstract class and interface?

→Interface has only abstract methods (Java 7), abstract class can have both abstract and concrete methods.

23. How do you implement an interface?

→Using implements keyword.

- **Outer and Inner Class**

24. What is an inner class?

→A class defined within another class.

25. Why use inner classes?

→To logically group classes that are only used in one place or to access private members of the outer class.

- **Custom Exception – DivisionByZero**

26. What is an exception?

→An event that disrupts the normal flow of a program.

27. How to create a user-defined exception?

→By extending Exception or RuntimeException class.

28. What is the use of try-catch-finally?

→try for risky code, catch to handle exception, finally always runs for cleanup.

- **Package – mypack**

29. What is a package?

→A namespace that organizes a set of related classes and interfaces.

30. How do you create a package?

→Using package mypack; at the top of the class.

31. How do you import a package?

→Using import mypack.ClassName;.

- **Thread using Runnable**

32. How do you create a thread using Runnable?

→ Implement Runnable interface, override run() method, then create Thread t = new Thread(obj) and start using t.start().

33. What is the use of start() method?

→ Starts a new thread and calls run() internally.

34. What does Thread.sleep(500) do?

→ Suspends the current thread for 500 milliseconds.

- **Thread using super()**

35. What is super() used for in threads?

→ It calls the constructor of the parent class Thread to name or configure the thread.

36. What is the difference between start() and run()?

→ start() creates a new thread; run() runs in the current thread.

37. How do main and child threads run together?

→ Both run concurrently after the child thread is started using start().

- **Multithreading General**

38. What is multithreading?

→ The ability to run multiple threads (tasks) concurrently.

39. What is synchronization?

→ It controls access to shared resources by multiple threads to avoid conflicts.

40. How do you communicate between threads?

→ Using wait(), notify(), and notifyAll() methods.

- **Enumerations and Wrappers**

41. What is an enumeration?

→ A special Java type that defines a collection of constants.

42. What is a wrapper class?

→A class that wraps a primitive type into an object.

Example: `int` → `Integer`.

43. What is autoboxing?

→Automatic conversion from primitive to wrapper.

Example: `int num = 5; Integer obj = num;`

44. What is unboxing?

→Conversion from wrapper to primitive.

Example: `Integer obj = 10; int num = obj;`

45. Explain OOPs concepts.

→Encapsulation, Inheritance, Polymorphism, Abstraction.

46. What is JVM?

→Java Virtual Machine — runs Java bytecode on any platform.

47. What is JDK and JRE?

→JDK — Java Development Kit (develop and run Java).

→JRE — Java Runtime Environment (only run Java).

48. What is exception propagation?

→When an exception is not caught in a method, it moves up the call stack.

49. Difference between throw and throws?

→`throw` is used to actually throw an exception.

→`throws` declares exceptions in the method signature.

50. What happens if we don't handle exceptions?

→The program terminates abnormally with an error message.



PAN-No. : AAATG6551G
GST-No. : 29AAATG6551G1ZL

GSSS

GEETHA SHISHU
SHIKSHANA SANGHA(R)

INSTITUTE OF ENGINEERING AND TECHNOLOGY FOR WOMEN

(Affiliated to Visvesvaraya Technological University, Belagavi. Approved by AICTE New Delhi & Govt. of Karnataka)

KRS ROAD, METAGALLI | MYSURU - 570 016 | KARNATAKA | INDIA



Phone : 0821 - 2581304,4257304 / 2977306 **Fax :** 0821 - 2581305 **Email :** gsssegg@gmail.com, principal@gsss.edu.in **Web :** www.geethashishu.in

Institute Vision

"To become a recognized world class Women Educational Institution, by imparting professional education to the students, creating technical opportunities through academic excellence and technical achievements, with ethical values"

Institute Mission

- To support value-based education with state of art infrastructure.
- To empower women with the additional skill for professional future career
- To enrich students with research blends in order to fulfil the international challenges
- To create multidisciplinary centre of excellence
- To achieve Accreditation standards towards international education recognition.
- To establish more Post Graduate & Research course.
- To increase Doctorates numbers towards the Research quality of academics.

